DBMS LAB — EXPERIMENT 10

Query Optimization on 10,000 Rows: Indexing, Query Rewriting and EXPLAIN ANALYZE

Note: These are expected/representative outputs based on the described SQL and sample data, not screenshots from a live MySQL run. Exact formatting, timings and EXPLAIN plans can vary by MySQL version.

Output 1: Verify dataset

Total_Rows	First_Order	Last_Order
10000	2026-01-01	2026-10-27

Output 2: Slow Query 1 – City filter (before)

With no index on City, all 10,000 rows are scanned to return the 2,000 Lucknow orders.

```
-> Filter: (customer_order.City = 'Lucknow')  (cost=1000 rows=1000) (actual time=0.026..1.98 rows=2000 loops=1)
    -> Table scan on Customer_Order  (cost=1000 rows=10000) (actual time=0.023..1.52 rows=10000 loops=1)
```

Output 3: Optimized Query 1 – idx_city (after)

```
-> Index lookup on Customer_Order using idx_city (City='Lucknow')  (cost=201 rows=2000) (actual
time=0.165..1.58 rows=2000 loops=1)
```

Output 4: Slow Query 2 – Customer and date filter (before)

Customer_ID = 1250 occurs in 10 rows, and 6 of them fall between 2026-02-01 and 2026-06-30. Without an index the full table is scanned to find them.

```
-> Filter: ((customer_order.Customer_ID = 1250) and (customer_order.Order_Date between '2026-02-01' and
'2026-06-30'))  (cost=1000 rows=111) (actual rows=6 loops=1)
    -> Table scan on Customer_Order  (cost=1000 rows=10000) (actual rows=10000 loops=1)
```

Output 5: Optimized Query 2 – idx_customer_date (after)

```
-> Index range scan on Customer_Order using idx_customer_date over (Customer_ID = 1250 AND '2026-02-01' <=
Order_Date <= '2026-06-30'), with index condition: (...)  (actual rows=6 loops=1)
```

Output 6: Slow Query 3 – Function on indexable column (before)

YEAR(Order_Date) = 2026 wraps the column in a function, so an index on Order_Date cannot be used and every row is evaluated. All 10,000 orders fall in 2026.

Total_Orders
10000

```
-> Aggregate: count(0)  (cost=2000 rows=1) (actual time=1.76..1.76 rows=1 loops=1)
    -> Filter: (year(customer_order.Order_Date) = 2026)  (cost=1000 rows=10000) (actual rows=10000 loops=1)
        -> Table scan on Customer_Order  (cost=1000 rows=10000) (actual rows=10000 loops=1)
```

Output 7: Optimized Query 3 – Date range rewrite (after)

The rewritten condition compares the bare column, so the optimizer can use an index on Order_Date. Because every row is in 2026, all 10,000 index entries still qualify; the gain here is a narrow covering-index scan instead of reading full rows, and the pattern gives large savings on selective ranges.

Total_Orders
10000

```
-> Aggregate: count(0)  (cost=2000 rows=1) (actual time=1.72..1.72 rows=1 loops=1)
    -> Filter: ((Order_Date >= DATE'2026-01-01') and (Order_Date < DATE'2027-01-01'))  (actual rows=10000
loops=1)
        -> Covering index scan on Customer_Order using idx_order_date  (cost=1000 rows=10000) (actual
rows=10000 loops=1)
```

Output 8: Slow Query 4 – Category and status filter (before)

Electronics and DELIVERED both occur when MOD(n,4) = 0, so 2,500 rows match. Without an index all 10,000 rows are scanned.

```
-> Filter: ((Order_Status = 'DELIVERED') and (Product_Category = 'Electronics'))  (cost=1000 rows=100) (actual
rows=2500 loops=1)
    -> Table scan on Customer_Order  (cost=1000 rows=10000) (actual rows=10000 loops=1)
```

Output 9: Optimized Query 4 – idx_category_status (after)

```
-> Index lookup on Customer_Order using idx_category_status (Product_Category='Electronics',
Order_Status='DELIVERED')  (cost=251 rows=2500) (actual rows=2500 loops=1)
```

Output 10: Slow Query 5 – Aggregation by city (before)

The query scans the whole table, aggregates in a temporary table, then sorts. Expected result:

City	Total_Sales
Jaipur	8625000.00
Noida	6700000.00
Kanpur	4875000.00
Lucknow	3150000.00
Delhi	1525000.00

```
-> Sort: Total_Sales DESC  (actual rows=5 loops=1)
    -> Table scan on <temporary>  (actual rows=5 loops=1)
        -> Aggregate using temporary table  (actual rows=5 loops=1)
            -> Table scan on Customer_Order  (cost=1000 rows=10000) (actual rows=10000 loops=1)
```

Output 11: Optimized Query 5 – covering index idx_city_total (after)

The result is identical. The covering index already stores City and Total_Amount in City order, so MySQL streams the groups and no temporary table is needed.

```
-> Sort: Total_Sales DESC  (actual rows=5 loops=1)
    -> Stream results  (cost=2000 rows=5) (actual rows=5 loops=1)
        -> Group aggregate: sum(customer_order.Total_Amount)  (actual rows=5 loops=1)
            -> Covering index scan on Customer_Order using idx_city_total  (cost=1000 rows=10000) (actual
rows=10000 loops=1)
```

Output 12: Final index verification

Key_name	Non_unique	Seq_in_index	Column_name	Index_type
PRIMARY	0	1	Order_ID	BTREE
idx_city	1	1	City	BTREE
idx_customer_date	1	1	Customer_ID	BTREE
idx_customer_date	1	2	Order_Date	BTREE
idx_order_date	1	1	Order_Date	BTREE
idx_category_status	1	1	Product_Category	BTREE
idx_category_status	1	2	Order_Status	BTREE
idx_city_total	1	1	City	BTREE
idx_city_total	1	2	Total_Amount	BTREE

Output 13: Before and after comparison

Query_Name	Before_Optimization	After_Optimization
Query 1 - City Filter	Before: no index on City	After: idx_city
Query 2 - Customer + Date	Before: no composite index	After: idx_customer_date
Query 3 - YEAR(Order_Date)	Before: function on column	After: date range rewrite
Query 4 - Category + Status	Before: no composite index	After: idx_category_status
Query 5 - City Aggregation	Before: no covering index	After: idx_city_total

Query	Access before	Access after	Rows read before	Rows read after
1	Full table scan	Index lookup	10000	2000
2	Full table scan	Index range scan	10000	6
3	Full table scan, function filter	Covering index scan	10000	10000
4	Full table scan	Index lookup	10000	2500
5	Full scan + temporary table	Covering index, group aggregate	10000	10000

Important

These are expected behaviors, not literal server-captured outputs. Row counts, timings, costs and plan wording must be checked against a live run of the exact SQL statements in your repository.